E-Commerce Platform

Complete Documentation Suite · Spring Boot · Kafka · Kubernetes · Floci (AWS)

Generated: 1/7/2026

Master Guide — Complete Project Reference

> The single document to learn this entire project: the exact order to read every file,
>
> high- and low-level design, all diagrams, every script, all code, and the full record
>
> of what was built and fixed.

PART A — The Learning Sequence (read files in THIS order)

Follow these 10 stages top to bottom. Each builds on the previous.

Stage 0 — See it run first

#	File / Action	Why
0.1	`docker compose up -d`	Watch the whole system boot before reading any code
0.2	`curl localhost:8000/api/products`	Prove the gateway → service path works

Stage 1 — The map

#	File	What you learn
1.1	`README.md`	Project overview, service list, quick start
1.2	`docs/MASTER_GUIDE.md` (this file)	The whole picture
1.3	`docs/ARCHITECTURE.md`	System diagrams + data flow

Stage 2 — How traffic enters

#	File	What you learn
2.1	`docker-compose.yml`	Every container, port, env var, dependency
2.2	`kong/kong.yml`	Gateway routing: which URL → which service (`strip_path:false`)

Stage 3 — One service, end to end (use `user-service` — simplest)

#	File	Layer
3.1	`user-service/src/main/resources/application.yml`	Config: DB, Redis, JWT
3.2	`.../model/User.java`	Entity (maps to `users` table)
3.3	`.../repository/UserRepository.java`	DB queries (Spring Data JPA)
3.4	`.../service/UserService.java`	Business logic

3.5	`.../controller/UserController.java`	REST endpoints
3.6	`.../config/SecurityConfig.java` + `JwtAuthenticationFilter.java`	JWT auth
3.7	`.../resources/db/migration/V1__init.sql`	Flyway schema

Stage 4 — The event backbone (`order` → `inventory` → `payment` → `notification`)

#	File	What you learn
4.1	`order-service/.../kafka/OrderEventPublisher.java`	Publishing `order.created`
4.2	`inventory-service/.../kafka/InventoryEventConsumer.java`	Reserving stock
4.3	`inventory-service/.../service/InventoryService.java`	Redis lock + atomic upsert
4.4	`payment-service/.../kafka/PaymentEventConsumer.java`	Charging only after stock OK
4.5	`order-service/.../kafka/OrderEventConsumer.java`	Completing saga → PAID / CANCELLED
4.6	`notification-service/.../kafka/NotificationConsumer.java`	Emails via SES

Stage 5 — Caching & concurrency

#	File	What you learn
5.1	`product-service/.../service/ProductService.java`	Redis cache (15-min TTL)
5.2	`inventory-service/.../service/InventoryService.java`	Redis distributed lock (oversell prevention)

Stage 6 — Containerisation

#	File	What you learn
6.1	`*/Dockerfile`	How each service becomes an image
6.2	`pom.xml` (root)	Multi-module reactor, Flyway, JaCoCo

Stage 7 — Kubernetes

#	File	What you learn

7.1	`k8s/namespace/namespace.yaml`	Namespaces
7.2	`k8s/postgres/postgres.yaml`	StatefulSets (5 DBs)
7.3	`k8s/kafka/kafka.yaml` + `redis/redis.yaml`	Stateful infra
7.4	`k8s/kong/kong.yaml`	Gateway + LoadBalancer service
7.5	`k8s/services/deployments.yaml`	Deployments + HPA
7.6	`k8s/services/configmap.yaml` + `secrets.example.yaml`	Config vs secrets
7.7	`k8s/monitoring/monitoring.yaml`	Prometheus, Grafana, Loki

Stage 8 — AWS via Floci (scripts)

#	File	What you learn
8.1	`scripts/01-vpc.sh`	VPC, subnets, SG, NAT
8.2	`scripts/02-alb.sh`	ALB, WAF, ACM, Route53
8.3	`scripts/03-ecr.sh`	Build + push images to ECR
8.4	`scripts/04-eks.sh`	EKS cluster + node groups
8.5	`scripts/05-deploy.sh`	Deploy manifests + register in ALB
8.6	`scripts/06-teardown.sh`	Delete everything

Stage 9 — CI/CD & governance

#	File	What you learn
9.1	`.github/workflows/ci-cd.yml`	Full pipeline
9.2	`.github/smoke-test.sh`	16-endpoint E2E saga test
9.3	`.github/workflows/codeql.yml`	Security scanning
9.4	`.github/dependabot.yml`	Dependency automation
9.5	`scripts/setup-github-governance.sh`	Environments + branch protection
9.6	`docs/CI_CD.md`, `DEPLOYMENT.md`, `AUTOSCALING.md`, `MONITORING.md`	Deep dives

PART B — High-Level Design (HLD)

HLD = the big picture: major components and how they talk. No class detail.

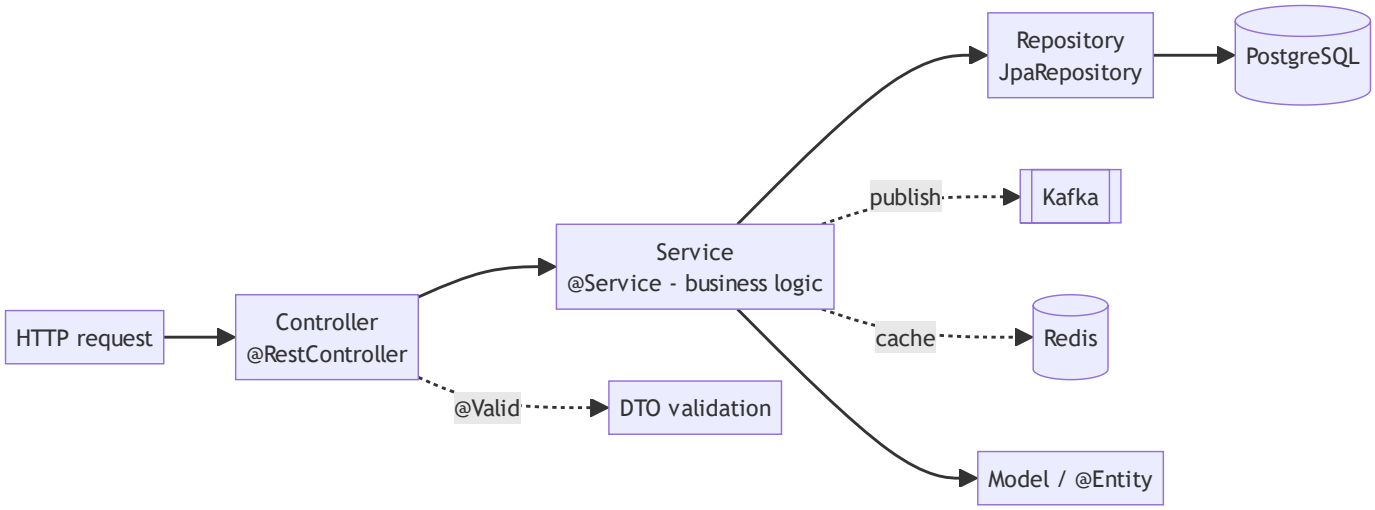
HLD principles

Principle	How it's applied
Database per service	5 separate Postgres DBs; no shared tables
API Gateway	Kong is the single entry; services never exposed directly
Event-driven	Services communicate via Kafka events, not direct calls (loose coupling)
Stateless services	Any pod can serve any request; state lives in DB/Redis/Kafka
CQRS-lite / Saga	Order flow is a choreographed saga across services

PART C — Low-Level Design (LLD)

LLD = the internals: layers, classes, tables, and exact message flow.

C1. Standard service layering (every service follows this)



Layer	Annotation	Responsibility	Must NOT
Controller	`@RestController`	HTTP mapping, status codes	contain business logic
Service	`@Service` `@Transactional`	business rules, orchestration	know about HTTP
Repository	`extends JpaRepository`	DB access only	contain logic
Model	`@Entity`	table mapping	contain behaviour
DTO	plain + `@Valid`	request/response shape	expose entities directly
Kafka	`@KafkaListener` / publisher	async events	block on responses

C2. Database schemas (LLD data model)

```
userdb.users            (id, name, email UNIQUE, password_hash, role, active, created_at)
productdb.products      (id, name, price, category, description, created_at)
orderdb.orders          (id, user_id, status, total_amount, shipping_address, created_at, upda
ted_at)
orderdb.order_items     (id, order_id FK, product_id, product_name, quantity, price)
inventorydb.inventory   (id, product_id UNIQUE, quantity, reserved, version)   ← @Version opti
mistic lock
paymentdb.payments      (id, order_id, user_id, amount, status, transaction_id, created_at)
```

C3. The Order Saga (detailed LLD sequence)

Order status machine: `PENDING → PAID` (happy) or `PENDING → CANCELLED` (no stock / payment fail).

C4. Kafka topics (LLD messaging)

Topic	Publisher	Consumers	Payload keys
`order.created`	order	inventory, payment*, notification	orderId, userId, amount, items
`inventory.updated`	inventory	order, payment	orderId, success, reason
`payment.processed`	payment	order, notification	orderId, success, transactionId
`order.cancelled`	order	inventory, notification	orderId, reason
`product.created`	product	inventory	productId

*payment only acts on `inventory.updated{success:true}`, never on raw `order.created`.

C5. Redis usage (LLD)

Key pattern	Service	Purpose	TTL
`session:{email}`	user	JWT session	24h
`product:{id}`	product	read cache	15 min

`inventory:lock:{productId}`	inventory	distributed lock (oversell guard)	30s

PART D — Every Script Explained

Script	Does	Key AWS concepts
`scripts/01-vpc.sh`	VPC 10.0.0.0/16, 2 public + 2 private subnets, IGW, NAT, route tables, 3 security groups	CIDR, public/private subnet, NAT, SG
`scripts/02-alb.sh`	ALB in public subnets, target group (IP mode), listeners 80→443, WAF, ACM cert, Route53	ALB, listeners, target groups, WAF, ACM
`scripts/03-ecr.sh`	Create 6 ECR repos, `mvn package`, `docker build`, push to Floci ECR (localhost:5100)	ECR, docker login, image tags
`scripts/04-eks.sh`	IAM roles, EKS cluster, 2 node groups, kubeconfig, **auto-writes k3s registries.yaml for ECR pulls**	EKS, IAM roles, node groups
`scripts/05-deploy.sh`	Apply all k8s manifests in order, register Kong pod IPs in ALB target group	kubectl apply order, target registration
`scripts/06-teardown.sh`	Delete EKS, ALB, WAF, VPC, ECR	clean teardown
`scripts/setup-github-governance.sh`	Create `production` + `dev` environments, branch protection on main/develop via GitHub API	environments, required reviewers, branch rules
`.github/smoke-test.sh`	Drive all 16 REST endpoints through Kong + verify order→PAID saga; runs local (KONG_URL) or CI (port-forward)	E2E testing
`scripts/generate-pdf-docs.js`	Combine all docs into one printable HTML	docs tooling

PART E — What We Built & Fixed (the full journey)

Phase 1 — Get endpoints working on Docker

Bug found	Fix
Kafka advertised `localhost:9092` in compose	changed to `kafka:9092`
Product Redis cache crash (serialization)	`Product implements Serializable`
Order JSON infinite recursion	`@JsonIgnore` on `OrderItem.order`
No JWT filter in user-service	added `JwtAuthenticationFilter` + stateless `SecurityConfig`
500s instead of proper codes	added `GlobalExceptionHandler` (404/401/409)
Kong 404 on all routes	added `strip_path: false` to all 6 routes
Payment charged even when out of stock	rewired: payment only on `inventory.updated{success:true}`

Phase 2 — Kubernetes

Bug	Fix
Kafka crash-loop (missing KAFKA_NODE_ID)	merged duplicate `env:` block
Kafka `$(POD_NAME)` unresolved	reordered env vars
Promtail crash (missing ConfigMap)	added `promtail-config` + RBAC
Secrets in git	removed; inject via `kubectl create secret`

Phase 3 — CI/CD

Bug	Fix
`configure-aws-credentials` rejects test creds	export env vars directly for Floci
Kind too slow for CI	switched to k3d (fast k3s)
k3d-action pinned v5.4.6 URL 404	install latest k3d directly
CI smoke test 503	wait for ALL services before testing

Trivy re-downloads DB	cached official action

Phase 4 — Completing the app (found via full E2E test)

Gap	Fix
No way to add stock (every order cancelled)	added `POST /api/inventory/{id}/restock`
Paid orders stuck at PENDING forever	added `payment.processed` listener → mark PAID
`restock` 500 under concurrency (duplicate key race)	atomic `INSERT ... ON CONFLICT` upsert

Phase 5 — Governance

- `production` environment with required reviewer (approval gate)
- Branch protection: PR + 6 test checks + no force-push on main & develop
- Repo made public to unlock free branch protection
- **Verified end-to-end:** develop auto-deploy green, main gated deploy approved & green

---

PART F — How To Run / Verify Everything

```
# 1. Local (fastest)
docker compose up -d
KONG_URL=http://localhost:8000 bash .github/smoke-test.sh   # expect 17/17
```

2. Full AWS simulation (Floci)

```
bash scripts/01-vpc.sh && bash scripts/02-alb.sh && bash scripts/03-ecr.sh
bash scripts/04-eks.sh && bash scripts/05-deploy.sh
kubectl get pods -n ecommerce
```

3. CI/CD

```
git push origin develop     # → tests + k8s deploy + smoke test
```

```
open PR develop→main → merge → approve production gate
```

4. Monitoring

```
kubectl port-forward svc/grafana 3000:3000 -n monitoring
```

Definition of done (all achieved): every endpoint tested on every deploy, full saga reaches PAID, real Kubernetes deploy green on both environments, production gated behind human approval.

Project Onboarding Guide

> Read this first. It tells you what every file is for, and the exact order to learn this project.

---

1. What Is This Project?

A production-grade e-commerce backend built as a **teaching platform**.

It uses the same tools and patterns as real production systems: Spring Boot microservices, Kafka event-driven architecture, Kong API Gateway, Kubernetes, and AWS-compatible deployment via Floci.

The goal is not to build a shopping app — it's to teach you how professional systems are designed, deployed, and operated.

---

2. The Repository Structure (What Every File Is For)

```
ecommerce/
│
├── README.md                       ← START HERE. Quick navigation + quick start.
├── docker-compose.yml              ← Runs the entire system locally with one command
├── pom.xml                         ← Parent Maven config — shared dependencies for all 6 servic
es
│
├── docs/                           ← All documentation (you are here)
│   ├── ONBOARDING.md               ← This file — read first
│   ├── ARCHITECTURE.md             ← System design diagrams + data model + security layers
│   ├── DEVELOPER_GUIDE.md          ← Day-to-day workflow: branch, PR, test, deploy
│   ├── DEPLOYMENT.md               ← How to deploy to Floci (simulated AWS)
│   ├── AUTOSCALING.md              ← How HPA (horizontal pod autoscaling) works
│   ├── MONITORING.md               ← Logs (Loki), metrics (Prometheus), dashboards (Grafana)
│   └── CI_CD.md                    ← GitHub Actions pipeline explained step by step
│
├── user-service/                   ← One folder per microservice (same structure in all 6)
├── product-service/
├── order-service/
├── inventory-service/
├── payment-service/
└── notification-service/
│
├── kong/
│   └── kong.yml                    ← Kong API Gateway routing rules (which URL goes to which se
rvice)
│
├── k8s/                            ← Kubernetes manifests (deploy to EKS/Floci)
│   ├── namespace/namespace.yaml    ← Creates 'ecommerce' and 'monitoring' namespaces
│   ├── postgres/postgres.yaml      ← 5× PostgreSQL StatefulSets (one per service)
│   ├── redis/redis.yaml            ← Redis StatefulSet
│   ├── kafka/kafka.yaml            ← Kafka StatefulSet (KRaft mode, no Zookeeper)
│   ├── kong/kong.yaml              ← Kong Deployment + ConfigMap with routing rules
│   ├── services/
│   │   ├── deployments.yaml        ← All 6 microservice Deployments + HPA
│   │   ├── configmap.yaml          ← Non-secret config (Kafka URL, Redis host, AWS endpoint)
│   │   └── secrets.example.yaml    ← Template for secrets (passwords, JWT key) — never committe
d
│   └── monitoring/monitoring.yaml ← Prometheus, Grafana, Loki, Promtail
│
├── scripts/                        ← Bash scripts to set up AWS-equivalent infra via Floci
│   ├── .env                        ← Auto-written by scripts; stores IDs (VPC, ALB, etc.)
│   ├── 01-vpc.sh                   ← VPC, subnets, IGW, NAT, route tables, security groups
│   ├── 02-alb.sh                   ← ALB, WAF, ACM cert, Route53, listeners, target group
│   ├── 03-ecr.sh                   ← ECR repos, build Docker images, push to Floci registry
│   ├── 04-eks.sh                   ← EKS cluster, IAM roles, node groups, kubeconfig
│   ├── 05-deploy.sh                ← Apply all K8s manifests, register Kong in ALB
│   ├── 06-teardown.sh              ← Delete everything (EKS, ALB, VPC, ECR)
│   └── README.md                   ← What each script does and AWS concepts covered
│
```

```
└── .github/
    ├── workflows/
    │   ├── ci-cd.yml            ← Main pipeline: test → build → scan → push → smoke test
    │   └── codeql.yml           ← Security scan (SAST) — runs on every PR
    └── dependabot.yml           ← Weekly automated dependency updates
```

---

3. Inside a Microservice (All 6 Are Identical in Structure)

Use `order-service/` as your reference:

```
order-service/
│
├── Dockerfile                                  ← How to build the Docker image
├── pom.xml                                     ← Maven dependencies for this service
│
└── src/
    ├── main/
    │   ├── java/com/ecommerce/orderservice/
    │   │   ├── OrderServiceApplication.java    ← Main class, starts Spring Boot
    │   │   ├── config/
    │   │   │   ├── SecurityConfig.java         ← JWT filter, which URLs need auth
    │   │   │   └── KafkaConfig.java            ← Kafka producer/consumer beans
    │   │   ├── controller/
    │   │   │   └── OrderController.java        ← REST endpoints (@GetMapping, @PostMappin
g)
    │   │   ├── service/
    │   │   │   └── OrderService.java           ← Business logic (create order, cancel orde
r)
    │   │   ├── repository/
    │   │   │   └── OrderRepository.java        ← Database queries (Spring Data JPA)
    │   │   ├── model/
    │   │   │   ├── Order.java                  ← @Entity — maps to 'orders' table
    │   │   │   └── OrderItem.java              ← @Entity — maps to 'order_items' table
    │   │   ├── dto/
    │   │   │   └── OrderRequest.java           ← What the API accepts (JSON body shape)
    │   │   ├── kafka/
    │   │   │   ├── OrderEventPublisher.java    ← Sends events to Kafka topics
    │   │   │   └── OrderEventConsumer.java     ← Receives events from Kafka
    │   │   └── exception/
    │   │       └── GlobalExceptionHandler.java ← Converts exceptions to HTTP responses
    │   │
    │   └── resources/
    │       ├── application.yml                 ← All config: DB URL, Kafka, Redis
    │       └── db/migration/
    │           └── V1__init.sql                ← Flyway: creates tables on first run
    │
    └── test/
        └── java/com/ecommerce/orderservice/
            └── OrderServiceTest.java           ← Unit tests (run in CI)
```

---

4. Recommended Learning Order

Follow this order. Each step builds on the previous one.

Step 1 — Run it first, understand it second

```
docker compose up -d

Wait 2 minutes, then:


curl http://localhost:8000/api/products
curl -X POST http://localhost:8000/api/users/register \
  -H "Content-Type: application/json" \
  -d '{"name":"You","email":"you@test.com","password":"test1234"}'
```

Before reading code, see the system working. This gives you the "what" before the "how."

Step 2 — Read the Architecture doc

`docs/ARCHITECTURE.md` — Read the system diagram. Understand the flow:

> Client → Kong → Service → Kafka → Downstream service → DB

This is the mental model everything else builds on.

Step 3 — Understand Kong routing

`kong/kong.yml` — This file tells Kong: "When someone calls `/api/orders/*`, forward to `order-service:8083`."

```
routes:
  - name: orders-route
    paths: ["/api/orders"]
    strip_path: false       # ← CRITICAL: keep /api/orders in the URL when forwarding
services:
  - name: order-service
    url: http://order-service:8083
```

Without Kong, clients would need to know the port of every service. Kong is the single front door.

Step 4 — Trace one request through the code

Follow a `POST /api/orders` from start to finish:

1. `kong/kong.yml` → Kong routes to `order-service:8083`
2. `order-service/controller/OrderController.java` → `@PostMapping("/api/orders")`
3. `order-service/service/OrderService.java` → validates, saves to DB, publishes event
4. `order-service/kafka/OrderEventPublisher.java` → sends `order.created` event to Kafka
5. `inventory-service/kafka/InventoryEventConsumer.java` → receives `order.created`
6. `inventory-service/service/InventoryService.java` → reserves stock, publishes `inventory.updated`
7. `payment-service/kafka/PaymentEventConsumer.java` → receives `inventory.updated` (if success)

8. `payment-service/service/PaymentService.java` → processes payment, publishes `payment.processed`
9. `notification-service/kafka/NotificationConsumer.java` → receives, sends email via SES

This is the **Saga pattern** — each service does its job, then publishes an event for the next.

Step 5 — Read the database schemas

`*/src/main/resources/db/migration/V1__init.sql` in each service. These SQL files are the ground truth of what data each service owns.

```
-- order-service V1__init.sql
CREATE TABLE orders (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL,
  status VARCHAR(20),          -- PENDING, CONFIRMED, CANCELLED
  total_amount DECIMAL(10,2),
  created_at TIMESTAMP
);
```

Notice: **no user table in orderdb, no product table in orderdb.**

Each service only knows its own data. To know a user's name, order-service would call user-service's API (or receive it in the Kafka event payload).

Step 6 — Read one service end-to-end

Read `user-service/` completely. It's the simplest (no Kafka consumers):

- `model/User.java` → understand the entity
- `repository/UserRepository.java` → understand how DB queries work
- `service/UserService.java` → understand the business logic
- `controller/UserController.java` → understand the REST endpoints
- `config/SecurityConfig.java` → understand JWT authentication
- `application.yml` → understand configuration

Then apply this understanding to other services.

Step 7 — Understand the CI/CD pipeline

`docs/CI_CD.md` → Then read `.github/workflows/ci-cd.yml`. Understand what happens automatically when you open a PR and when you merge.

Step 8 — Deploy to Kubernetes via Floci

`docs/DEPLOYMENT.md` → Then run `scripts/01-vpc.sh` through `scripts/05-deploy.sh`. This teaches you AWS networking, EKS, ECR, ALB — with real CLI commands.

Step 9 — Explore monitoring

`docs/MONITORING.md` → Then run:

```
kubectl port-forward svc/grafana 3000:3000 -n monitoring
open http://localhost:3000
```

Explore logs (Loki) and metrics (Prometheus) from your running services.

---

5. Key Concepts — What You'll Learn Here

Concept	Where to See It	Doc
REST API design	`*/controller/*.java`	—
Spring Security + JWT	`user-service/config/SecurityConfig.java`	—
Event-driven architecture	`*/kafka/` in each service	ARCHITECTURE.md
Saga pattern	order → inventory → payment flow	ARCHITECTURE.md
Database-per-service	5 separate `postgres-*` in docker-compose	ARCHITECTURE.md
Redis caching	`product-service/service/ProductService.java`	—
Redis distributed locking	`inventory-service/service/InventoryService.java`	—
Flyway migrations	`*/resources/db/migration/V1__init.sql`	—
API Gateway pattern	`kong/kong.yml`	—
Docker multi-service	`docker-compose.yml`	—
Kubernetes deployments	`k8s/services/deployments.yaml`	DEPLOYMENT.md
HPA autoscaling	`k8s/services/deployments.yaml` (HPA blocks)	AUTOSCALING.md
CI/CD pipeline	`.github/workflows/ci-cd.yml`	CI_CD.md
VPC networking	`scripts/01-vpc.sh`	DEPLOYMENT.md
Load balancing (ALB)	`scripts/02-alb.sh`	DEPLOYMENT.md
Container registry (ECR)	`scripts/03-ecr.sh`	DEPLOYMENT.md
Managed Kubernetes (EKS)	`scripts/04-eks.sh`	DEPLOYMENT.md
Security scanning	`.github/workflows/ci-cd.yml` (Trivy, CodeQL)	CI_CD.md

Observability	`k8s/monitoring/monitoring.yaml`	MONITORING.md

---

6. Common Mistakes to Avoid

"Why is Kong returning 404?" Check `kong/kong.yml`. Every route must have `strip_path: false` or Kong will strip `/api/orders` from the URL before forwarding, and the service won't match any endpoint. **"Why is the order-service calling payment even when out of stock?"** Read `docs/ARCHITECTURE.md` → Kafka Event Flow. Inventory must publish `inventory.updated {success: true}` first. Payment only listens to that event, not `order.created`. **"Why does the service crash on startup?"**

1. Flyway can't connect to Postgres yet — look for `connect-retries: 15` in `application.yml`. Postgres takes 5-10s to start.
2. Kafka admin bean fails — `spring.kafka.admin.fail-fast: false` in `application.yml` prevents this.

"My changes aren't in the Docker image."

```
# After changing code, rebuild:
docker compose up -d --build user-service
```

"I pushed to main directly." Branch protection should prevent this. Always: feature branch → PR → develop → PR → main.

---

7. Your First Contribution — Step by Step

```
# 1. Clone and start
git clone https://github.com/SumedhDhamdhere/ecommerce-platform
cd ecommerce-platform
docker compose up -d

Wait ~2 min for all services to start
```

2. Verify it works

```
curl http://localhost:8000/api/products
```

3. Create your feature branch

```
git checkout develop
git pull
git checkout -b feature/your-name-first-task
```

4. Make a small change (e.g., add a field to ProductController)

Edit the file, save

5. Run tests for that service

```
cd product-service
mvn test
```

6. Test your change manually

```
curl http://localhost:8000/api/products
```

7. Push and open a PR

```
git add .
git commit -m "feat: add stock field to product response"
git push -u origin feature/your-name-first-task
```

Open PR on GitHub → target: develop

8. Watch CI run (Tests + Coverage + SAST)

If all green → request review from tech lead

```
After approval → merge to develop → watch CI deploy automatically
```

---

8. Getting Help

- **Something broken?** Check `docker compose logs` first.
- **K8s pod crashing?** `kubectl describe pod -n ecommerce`
- **CI pipeline confused?** Read `.github/workflows/ci-cd.yml` and `docs/CI_CD.md`
- **Architecture question?** `docs/ARCHITECTURE.md`
- **Deployment question?** `docs/DEPLOYMENT.md` + `scripts/README.md`

Architecture

System Overview

Kafka Event Flow (Order Saga)

Key design decision: Payment only fires AFTER inventory confirms stock is reserved. If inventory fails → order is automatically cancelled → no payment taken.

Data Model (DB per service)

Each service owns its own PostgreSQL database. No cross-service DB queries.

```
userdb        → users (id, email, password_hash, role, active)
productdb     → products (id, name, price, category, stock=in inventory)
orderdb       → orders + order_items
inventorydb   → inventory (product_id, quantity, reserved, version)
paymentdb     → payments (order_id, amount, status, transaction_id)
```

Redis Usage (3 purposes)

Purpose	Service	Key pattern	TTL
JWT sessions	user-service	`session:`	24h
Product cache	product-service	`product:`	15 min
Distributed lock	inventory-service	`inventory:lock:`	30s

Security Layers

```
Internet
  ↓
WAF (OWASP rules + rate limit 2000 req/5min + SQLi blocking)
  ↓
ALB (SSL/TLS termination — HTTPS only)
  ↓
Kong (rate limiting 1000/min + correlation IDs + JWT validation)
  ↓
Services (Spring Security + JWT + input validation)
  ↓
Databases (separate credentials per service, no cross-service access)
```

Floci vs Real AWS

Component	Floci (local)	Real AWS

VPC / Subnets / SGs	✅ Full API simulation	AWS EC2
ALB + listeners	✅ Port 80 redirect works; port 443 TLS not implemented	AWS ELB
WAF	✅ API works	AWS WAF
ACM (SSL cert)	✅ API works	AWS ACM
Route 53	✅ API works	AWS Route 53
ECR (push/pull)	✅ Real Docker Registry v2 at localhost:5100	AWS ECR
EKS (cluster)	✅ Real k3s cluster provisions	AWS EKS
SES (email)	✅ Real API, emails logged	AWS SES
S3	✅ Full API	AWS S3
CloudWatch Logs	✅ API works	AWS CloudWatch

Developer Guide

Prerequisites

Tool	Version	Install
Docker Desktop	Latest	docker.com
Java JDK 17	17 LTS	`brew install openjdk@17`
Maven	3.9+	`brew install maven`
Git	Any	git-scm.com
AWS CLI	v2	`pip install awscli`

Day-to-Day Workflow

1. Start Working on a Feature

```
# Always branch off develop, never off main
git checkout develop
git pull
git checkout -b feature/add-coupon-codes
```

2. Make Changes and Test Locally

```
# Start all infrastructure
docker compose up -d

# Run tests for the service you changed
cd order-service
mvn test
```

```
Test your endpoint manually

curl -X POST http://localhost:8000/api/orders ...
```

3. Push and Open a PR

```
git push -u origin feature/add-coupon-codes
```

```
Open PR on GitHub → targeting develop branch
```

What happens automatically when you open the PR:

- ✅ All 6 services' unit tests run (30 tests)
- ✅ JaCoCo coverage check (minimum 50%)
- ✅ CodeQL SAST scan (security analysis)
- ✅ PR cannot merge until tests pass

4. After PR is Reviewed and Merged to `develop`

What happens automatically:

- ✅ Tests run again
- ✅ Docker images built (all 6 services)
- ✅ Trivy vulnerability scan
- ✅ Images pushed to Floci ECR
- ✅ Full stack deployed via docker-compose
- ✅ Smoke test: register user → create product → place order
- ✅ Slack/email notification (if configured)

5. Promote to Production (merge develop → main)

```
# Open PR: develop → main on GitHub
```

```
After review + approval → merge
```

What happens:

- Same pipeline as develop
- PAUSES for manual approval (production GitHub Environment)
- Approver clicks "Approve deployment" in GitHub
- Then deploys

Branch Rules

```
main      ← production, protected, requires PR + approval
develop   ← staging, protected, requires PR
feature/* ← your work, branch off develop
hotfix/*  ← urgent fixes, branch off main
```

Never push directly to `main` or `develop`.

Running Services Locally (without Docker)

If you want to run services with hot-reload for development:

```
# Terminal 1: Start infrastructure only
docker compose up -d postgres-user postgres-product postgres-order \
  postgres-inventory postgres-payment redis kafka kafka-init kong floci

# Terminal 2+: Run a specific service
cd user-service
mvn spring-boot:run
```

```
Changes reload automatically with Spring DevTools
```

Environment Variables

All configuration is in `docker-compose.yml` for local development.

For K8s, see `k8s/services/configmap.yaml` and `k8s/services/secrets.example.yaml`.

Key values for local:

```
SPRING_DATA_REDIS_PASSWORD=redis_pass
SPRING_KAFKA_BOOTSTRAP_SERVERS=kafka:9092
```

```
DB credentials: see docker-compose.yml per service
```

Running the AWS Simulation (Floci)

```
# Configure AWS CLI to point to Floci
export AWS_ENDPOINT_URL=http://localhost:4566
export AWS_DEFAULT_REGION=ap-south-1
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test

# Now any aws CLI command goes to Floci instead o
f real AWS

aws s3 ls
aws ecr describe-repositories
aws eks list-clusters
```

Common Commands

```
# Check all pods in K8s
kubectl get pods -n ecommerce

# Check logs of a service

kubectl logs -n ecommerce -l app=order-service --tail=50
```

Check Kafka consumer lag

```
docker exec kafka kafka-consumer-groups \
  --bootstrap-server kafka:9092 \
  --describe --group payment-service-group
```

Connect to a database directly

```
docker exec -it postgres-order psql -U order_svc -d orderdb
```

Check Redis cache

```
docker exec redis redis-cli -a redis_pass KEYS '*'
```

Deployment Guide — Floci (Local AWS Simulation)

What is Floci?

Floci is a local AWS simulator. It runs on `localhost:4566` and accepts the exact same API calls as real AWS — `aws eks create-cluster`, `aws ecr push`, `aws elbv2 create-load-balancer` — but instead of billing you and provisioning real cloud resources, it spins up local containers.

Why use it? You learn every AWS concept (VPC, subnets, security groups, ALB, EKS, ECR, SES) with real CLI commands, zero cost, zero risk.

Architecture: What Gets Deployed Where

```
Your Machine
├── Floci (localhost:4566)        ← simulates AWS control plane
│   ├── VPC + Subnets + SGs      ← networking (scripts/01)
│   ├── ALB + WAF + Route53      ← edge layer (scripts/02)
│   ├── ECR Registry (localhost:5100)  ← Docker images (scripts/03)
│   └── EKS (real k3s cluster)   ← runs your pods (scripts/04-05)
│
└── k3s Kubernetes cluster        ← provisioned BY Floci
    ├── ecommerce namespace
    │   ├── 6 Spring Boot pods
    │   ├── Kong, Kafka, Redis, 5x Postgres
    │   └── HPA controllers
    └── monitoring namespace
        ├── Prometheus + Grafana
        └── Loki + Promtail
```

Prerequisites

```
# Verify these are installed
docker --version          # Docker Desktop running
aws --version             # AWS CLI v2
kubectl version --client  # kubectl
bash --version            # bash 4+
```

Step-by-Step Deployment

Step 0 — Start Floci

```
docker compose up -d floci

Wait until healthy:


curl http://localhost:4566/_floci/health
```

Step 1 — VPC + Networking (01-vpc.sh)

```
bash scripts/01-vpc.sh
```

What it creates:

- VPC `10.0.0.0/16`
- 2 public subnets (10.0.1.0/24, 10.0.2.0/24) — for ALB
- 2 private subnets (10.0.10.0/24, 10.0.11.0/24) — for pods/nodes
- Internet Gateway → attached to VPC
- NAT Gateway → in public subnet, routes private subnet traffic out
- Route tables → public routes to IGW, private routes to NAT
- Security groups:

- `alb-sg` : inbound 80, 443 from internet - `app-sg` : inbound 8000 from alb-sg only - `eks-sg` : inbound from app-sg only **AWS concepts taught:** CIDR, public vs private subnet, IGW, NAT, route tables, security groups as firewalls.

Step 2 — ALB + WAF + SSL (02-alb.sh)

```
bash scripts/02-alb.sh
```

What it creates:

- ALB in the 2 public subnets
- Target group (IP mode — targets are pod IPs, not node IPs)
- Listener 80 → redirect to 443
- Listener 443 → forward to target group
- WAF WebACL: OWASP rules + rate limit 2000 req/5min
- ACM certificate (self-signed in Floci)
- Route 53 hosted zone + alias record

AWS concepts taught: ALB vs NLB, listener rules, target groups, WAF, ACM, Route 53 alias records.

Step 3 — Build + Push Docker Images (03-ecr.sh)

```
bash scripts/03-ecr.sh
```

What it does:

1. Creates 6 ECR repositories in Floci (`ecommerce/user-service` , etc.)
2. Builds each Spring Boot service: `mvn package` → `docker build`
3. Tags images as `localhost:5100/ecommerce/:latest`
4. Pushes to Floci ECR (real Docker Registry v2 at localhost:5100)

AWS concepts taught: ECR (Elastic Container Registry), `docker login` with ECR token, image tagging, registry URL format.

```
# Verify images are in Floci ECR
aws ecr list-images --repository-name ecommerce/user-service \
  --endpoint-url http://localhost:4566
```

Step 4 — EKS Cluster (04-eks.sh)

```
bash scripts/04-eks.sh
```

What it creates:

- IAM role for EKS cluster (`eks-cluster-role`)
- IAM role for EKS nodes (`eks-node-role`) with ECR read access
- EKS cluster `ecommerce-cluster` in private subnets
- Node group `general-nodes` (m5.large × 3) — user, product, order, Kong
- Node group `high-mem-nodes` (r5.large × 2) — Kafka consumers, payment
- Updates your `~/.kube/config` so `kubectl` connects to this cluster

AWS concepts taught: EKS control plane vs nodes, IAM roles for EKS, managed node groups, instance types.

```
# Verify cluster is running
kubectl get nodes
kubectl cluster-info
```

IMPORTANT — Floci ECR image pull fix (one-time manual step): Floci's k3s node needs to know to pull from `localhost:5100` instead of Docker Hub:

```
# This is done automatically by 04-eks.sh now:

# Writes /etc/rancher/k3s/registries.yaml inside k3s container

CONTAINER=$(docker ps --filter "name=floci-eks" -q | head -1)
docker exec $CONTAINER bash -c 'mkdir -p /etc/rancher/k3s && cat > /etc/rancher/k3s/registrie
s.yaml << EOF
mirrors:
  "localhost:5100":
    endpoint:
      - "http://floci-ecr-registry:5000"
EOF
kill -SIGHUP 1'
```

Step 5 — Deploy to Kubernetes (05-deploy.sh)

```
bash scripts/05-deploy.sh
```

What it deploys (in order):

1. Namespaces (`ecommerce`, `monitoring`)
2. ConfigMaps + Secrets
3. StatefulSets: 5× Postgres, Redis, Kafka
4. Kong API Gateway
5. 6× Spring Boot microservice Deployments
6. Registers Kong pod IPs in ALB Target Group
7. Monitoring: Prometheus, Grafana, Loki, Promtail

```
# Verify everything is running
kubectl get pods -n ecommerce
kubectl get pods -n monitoring
kubectl get svc -n ecommerce


# Check ALB target health


aws elbv2 describe-target-health --target-group-arn $TG_ARN \
  --endpoint-url http://localhost:4566
```

Step 6 — Test the Deployment

```
# Get ALB DNS (from .env written by 02-alb.sh)
source scripts/.env
echo $ALB_DNS


# Test via ALB (goes: ALB → Kong → service)


curl http://$ALB_DNS/api/products
curl -X POST http://$ALB_DNS/api/users/register \
  -H "Content-Type: application/json" \
  -d '{"name":"Rahul","email":"r@test.com","password":"pass1234"}'
```

Teardown

```
bash scripts/06-teardown.sh
```

```
Deletes EKS cluster, node groups, ALB, WAF, VPC, ECR repositories
```

Troubleshooting

Problem	Check	Fix
Pod `ImagePullBackOff`	`kubectl describe pod -n ecommerce`	See registries.yaml fix in Step 4
Kong 503	`kubectl logs -n ecommerce -l app=kong`	Check if upstream service pod is running
ALB targets unhealthy	`aws elbv2 describe-target-health ...`	Pods not ready yet, wait 2min
Kafka pod crash-loop	`kubectl logs -n ecommerce kafka-0`	Check for KAFKA_NODE_ID env var issues
Flyway fails on startup	Service logs: `flyway connect-retry`	Postgres not ready; increase timeout

Local docker-compose vs Floci K8s

	docker-compose	Floci K8s
Start time	60s	5-10 min
Resource usage	Low	Medium
AWS concepts	None	Full (VPC, ALB, ECR, EKS)
Best for	Development	Learning deployment

Autoscaling — HPA (Horizontal Pod Autoscaler)

What is Autoscaling?

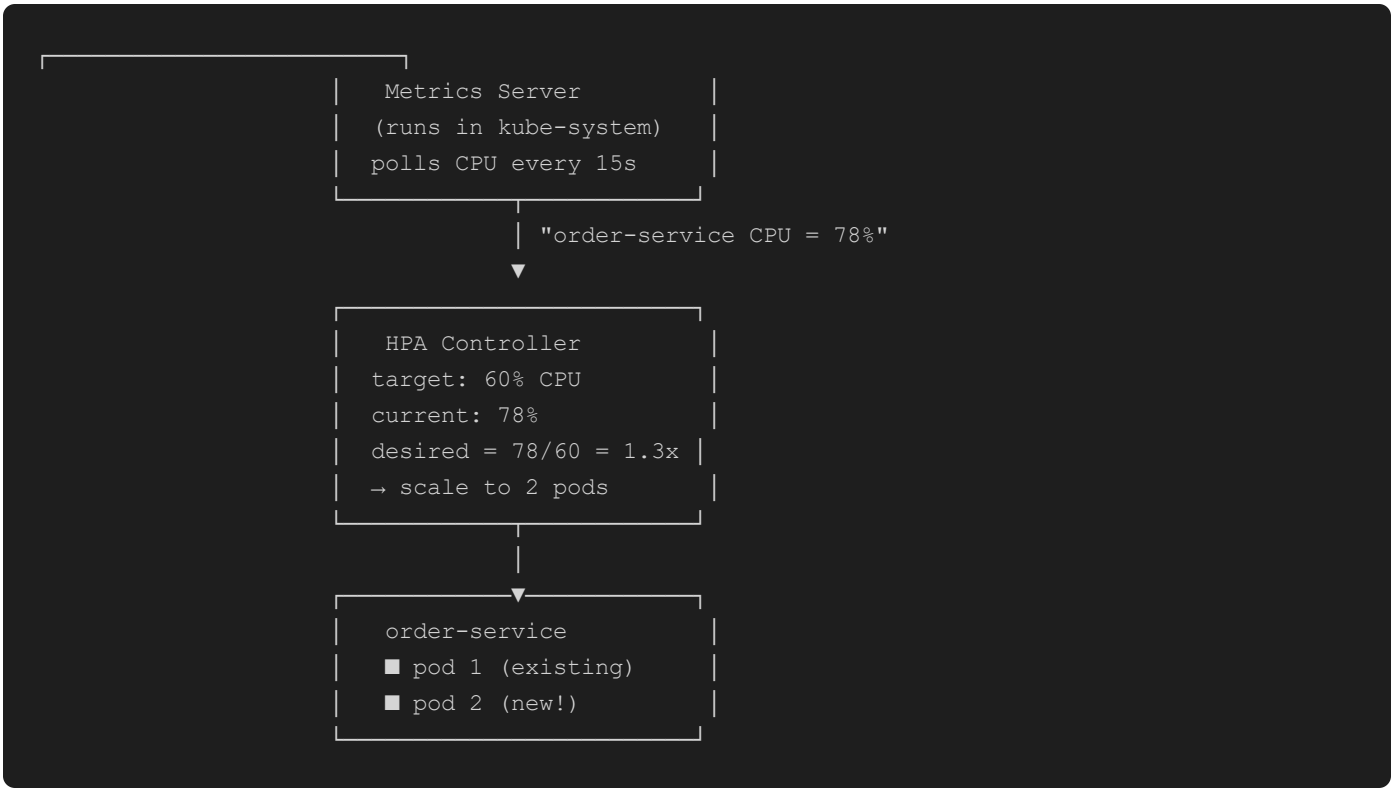
When traffic increases, Kubernetes automatically adds more copies (pods) of your service.

When traffic drops, it removes them. You pay only for what you use.

This is called **Horizontal Pod Autoscaling (HPA)** — "horizontal" means more pods,

not a bigger pod.

How HPA Works



Formula: `desired_replicas = ceil(current_replicas × current_metric / target_metric)`

Our HPA Configuration

From `k8s/services/deployments.yaml`, every microservice has:

```
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: order-service-hpa
  namespace: ecommerce
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: order-service
  minReplicas: 1       # never go below 1 (no cold starts)
  maxReplicas: 3       # never exceed 3 (resource limit)
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60    # scale when avg CPU > 60%
```

Setting	Value	Why
`minReplicas: 1`	Always 1 pod	Pod is ready immediately for requests
`maxReplicas: 3`	Cap at 3 pods	1 pod per service = learning, not production scale
`averageUtilization: 60%`	Scale at 60% CPU	Buffer before saturation; 80% would leave no headroom

---

Timeline: What Happens During a Traffic Spike

```
Time 0s   → Traffic increases 3×
Time 15s  → Metrics Server reports: CPU = 75%
Time 30s  → HPA calculates: need 2 pods. Sends scale command to Deployment.
Time 60s  → New pod scheduled on a node
Time 90s  → New pod passes readiness probe (Spring Boot started)
Time 90s  → Kong starts sending traffic to both pods (load balanced)
Time ...  → Traffic drops. HPA waits 5 minutes before scaling DOWN
            (scaleDown stabilization window — avoids flapping)
Time +5m  → HPA removes extra pod
```

Why 5 minutes before scale-down? Traffic often spikes and drops repeatedly. Removing a pod immediately and adding it back 30s later is wasteful and causes latency. The stabilization window prevents this "flapping."

Load Balancing Between Pods

Kong (the API Gateway) uses **round-robin** load balancing by default.

When order-service scales from 1 to 2 pods, Kong's upstream automatically picks up the new pod IP (via Kubernetes service DNS).

```
Client → Kong :8000 → order-service (K8s Service)
                          ├── pod-1 :8083  ← request 1
                          ├── pod-2 :8083  ← request 2
                          └── pod-1 :8083  ← request 3 (round-robin)
```

The **Kubernetes Service** (ClusterIP) is the stable DNS name (`order-service.ecommerce.svc.cluster.local`).

Kong always calls this DNS — Kubernetes itself load-balances across whichever pods are healthy.

Testing Autoscaling

```
# Watch HPA status live
kubectl get hpa -n ecommerce --watch


# Generate load on order-service


kubectl run load-test --image=busybox --rm -it --restart=Never -- \
  sh -c "while true; do wget -q -O- http://order-service.ecommerce.svc.cluster.local:8083/api/orders/user/1; done"
```

In another terminal — watch pods scale

```
kubectl get pods -n ecommerce --watch
```

Check CPU usage

```
kubectl top pods -n ecommerce
```

Expected output:

```
NAME                          READY   STATUS    REPLICAS
order-service-hpa             1/3     Running   1        ← initial
order-service-hpa             1/3     Running   2        ← after spike
order-service-hpa             1/3     Running   1        ← after quiet period
```

On Real AWS — What Changes

In production EKS, autoscaling has two more layers:

```
HPA              — adds/removes pods within existing nodes
Cluster Autoscaler — adds/removes EC2 nodes when pods can't be scheduled
  (triggered when: HPA wants 4 pods, but only 3 fit on existing nodes)


AWS Cost Implication:


  m5.large = ~$0.096/hr
  r5.large = ~$0.126/hr
  With our maxReplicas=3, worst case = 18 pods across ~6 nodes ≈ $0.60/hr
```

In Floci, the node groups are simulated — the HPA still runs and scales pods,

but there's no actual EC2 being added (k3s has unlimited virtual capacity).

Resource Requests vs Limits

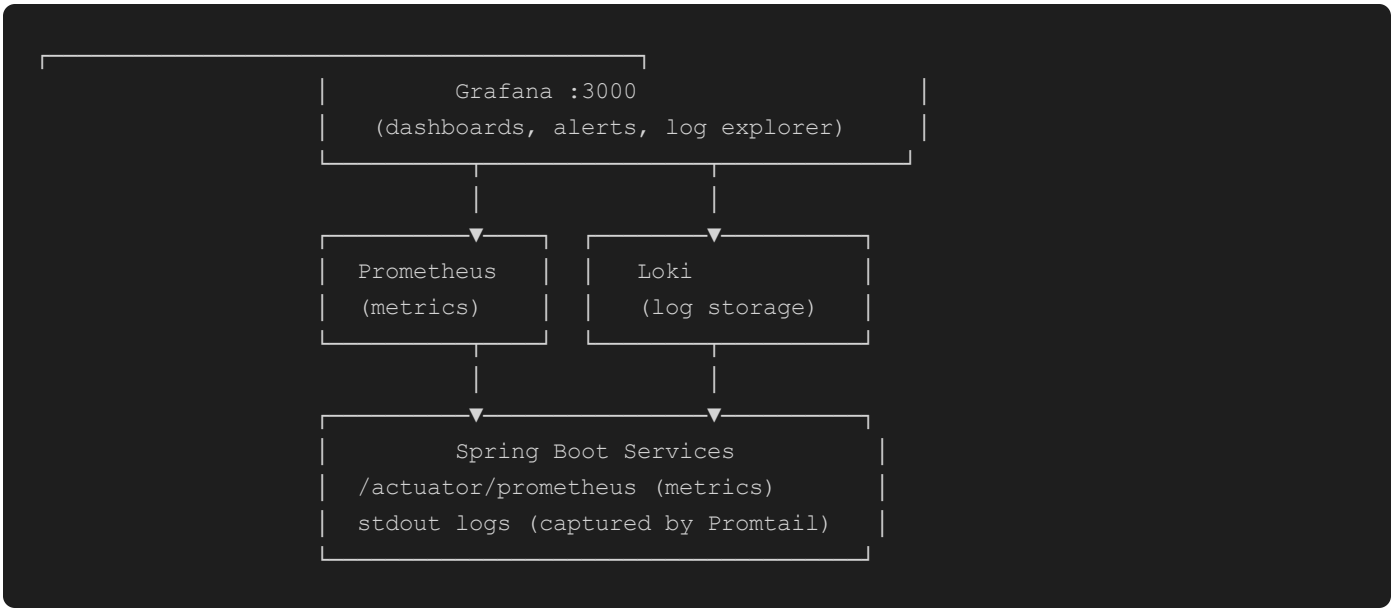
Every pod in `k8s/services/deployments.yaml` has:

```
resources:
  requests:
    cpu: 250m          # guaranteed: 0.25 CPU core
    memory: 512Mi      # guaranteed: 512MB RAM
  limits:
    cpu: 500m          # max: 0.5 CPU core (throttled if exceeded)
    memory: 1Gi        # max: 1GB RAM (OOM-killed if exceeded)
```

Why this matters for HPA: HPA calculates "% of request" not "% of node." If a pod uses 300m CPU and its request is 250m → usage = 120% → HPA scales immediately. Setting realistic requests is critical for HPA to work correctly.

Monitoring & Logs

Stack Overview



Tool	Port	Role	AWS Equivalent
Prometheus	9090	Scrapes metrics every 15s	CloudWatch Metrics
Grafana	3000	Dashboards + alerts	CloudWatch Dashboards
Loki	3100	Stores log lines	CloudWatch Logs
Promtail	DaemonSet	Ships logs to Loki	CloudWatch Agent

Accessing Grafana

```
# Port-forward from K8s
kubectl port-forward svc/grafana 3000:3000 -n monitoring

# Open in browser

open http://localhost:3000
```

```
Default login: admin / admin
```

Metrics (Prometheus)

Every Spring Boot service exposes metrics at `/actuator/prometheus`.

Prometheus scrapes them every 15s. Example metrics:

```
# HTTP request rate
http_server_requests_seconds_count{uri="/api/orders",status="200"}

# JVM memory
jvm_memory_used_bytes{area="heap"}
```

Kafka consumer lag

```
kafka_consumer_records_lag{topic="order.created",partition="0"}
```

```
# HikariCP (DB connection pool)

hikaricp_connections_active{pool="HikariPool-order-service"}
```

Key Grafana dashboards to set up:

- JVM Micrometer (Dashboard ID: 4701 — import from grafana.com)
- Spring Boot Statistics (Dashboard ID: 12900)
- Kafka Consumer Lag (Dashboard ID: 12554)

Logs (Loki + Promtail)

How logs flow:

```
Pod stdout → Promtail DaemonSet → Loki → Grafana (query with LogQL)
```

Promtail runs as a **DaemonSet** (one pod per Kubernetes node).

It watches `/var/log/pods/ecommerce_*/` and ships every log line to Loki.

Querying Logs in Grafana

Go to **Explore** → select **Loki** data source.

```
# All logs from order-service
{namespace="ecommerce", app="order-service"}

# Only ERROR lines

{namespace="ecommerce"} |= "ERROR"
```

Kafka-related logs

```
{namespace="ecommerce", app="order-service"} |= "order.created"
```

Failed payment events

```
{namespace="ecommerce", app="payment-service"} |= "payment.failed"
```

```
Last 100 lines from user-service
```

```
{app="user-service", namespace="ecommerce"} | limit 100
```

Correlation IDs (Tracing Requests)

Kong adds a `X-Request-ID` header to every request.

Spring Boot services log this ID. You can trace a single request across all services:

```
# Make a request
curl -H "Content-Type: application/json" \
  -X POST http://localhost:8000/api/orders ... \
  -v 2>&1 | grep "X-Request-ID"
```

```
→ X-Request-ID: a1b2c3d4-e5f6
```

Then search Loki for all logs with that ID

```
{namespace="ecommerce"} |= "a1b2c3d4-e5f6"
```

```
Shows the full journey: kong → order → kafka → inventory → payment → notification
```

Alerting

In Grafana, create alert rules under **Alerting → Alert Rules**:

```
Alert: High Error Rate
  Condition: rate(http_server_requests_seconds_count{status=~"5.."}[5m]) > 0.1
  For: 2 minutes
  → Send to: Slack channel / email

Alert: Pod Down

  Condition: kube_deployment_status_replicas_available{namespace="ecommerce"} < 1
  For: 1 minute
  → Send to: PagerDuty / Slack

Alert: Kafka Consumer Lag

  Condition: kafka_consumer_records_lag > 1000
  For: 5 minutes
  → Payment or inventory consumers are falling behind
```

---

vs CloudWatch (Real AWS)

This stack is our replacement for CloudWatch. Feature mapping:

Floci Stack	Real AWS
Promtail ships logs to Loki	CloudWatch Agent ships logs to CloudWatch Logs
`{app="order-service"}` in LogQL	`/ecommerce/order-service` log group in CloudWatch
Prometheus metrics in Grafana	CloudWatch Metrics + dashboards
Grafana alerts	CloudWatch Alarms → SNS → email/Lambda
Loki retention: 30 days (configurable)	CloudWatch Logs retention: 1 day to 10 years

Key difference: On real AWS, you'd add this to your `application.yml`:

```
management:
  cloudwatch:
    metrics:
      export:
        namespace: Ecommerce/OrderService
        enabled: true
```

Our setup achieves the same observability locally with no AWS costs.

Checking Logs Without Grafana (Quick Debug)

```
# Stream logs live
kubectl logs -n ecommerce -l app=order-service -f


Last 100 lines


kubectl logs -n ecommerce -l app=order-service --tail=100
```

Logs since a time

```
kubectl logs -n ecommerce -l app=order-service --since=10m
```

All services at once (requires kubetail or stern)

```
kubectl logs -n ecommerce -l tier=app -f --prefix
```

```
docker-compose equivalent

docker compose logs -f order-service --tail=50
```

Health Checks

All Spring Boot services expose:

```
# Is the service up?
curl http://localhost:8083/actuator/health

→ {"status":"UP","components":{"db":{"statu
s":"UP"},"kafka":{"status":"UP"}}}
```

Is it ready for traffic?

```
curl http://localhost:8083/actuator/health/readiness
```

```
# Detailed metrics endpoint (scraped by Prometheus)

curl http://localhost:8083/actuator/prometheus | head -30
```

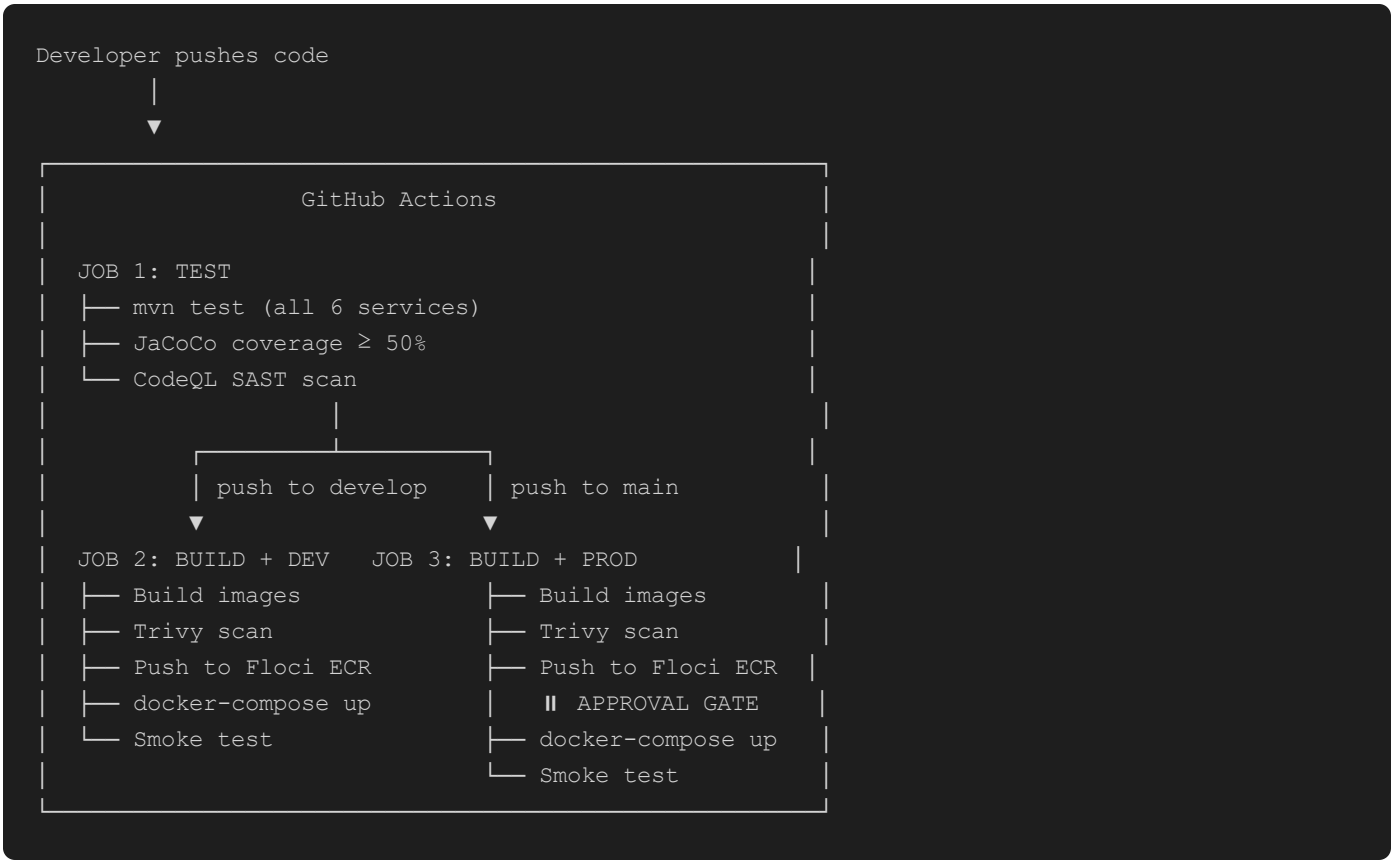
In Kubernetes, the readiness probe on each pod uses `/actuator/health/readiness`.

Kong only routes to a pod after its readiness probe passes. This ensures zero-downtime deployments — new pods only receive traffic when fully started.

CI/CD Pipeline

Overview

Every code change goes through automated testing, security scanning, image building, and deployment before it ever runs in "production."



Job 1: Tests (runs on every PR and push)

File: `.github/workflows/ci-cd.yml` → `test` job

```
services:
  postgres: postgres:15    # Real DB for integration tests
  redis: redis:7-alpine    # Real Redis for cache tests
```

Why real services in CI? Mocks lie. We run tests against actual Postgres and Redis so that if the DB schema is wrong or a query fails, CI catches it — not production. **JaCoCo coverage gate:**

```
mvn test
→ generates: target/site/jacoco/index.html
→ fails if line coverage < 50%
→ excludes: dto/, model/, config/, exception/ (boilerplate)
```

CodeQL (SAST):

- Scans for SQL injection, XSS, insecure deserialization
- `continue-on-error: true` — private repos don't get Advanced Security for free; scan still runs, results go to Security tab

---

Job 2: Build + Push + Deploy (develop branch)

Triggered only when code is **pushed to** `develop` (not on PRs).

Step 1 — Start Floci

```
docker compose up -d floci
```

```
Wait for: curl http://localhost:4566/_floci/health
```

Floci is the AWS simulator. ECR, SES, ALB APIs all respond at `localhost:4566`.

Step 2 — Configure AWS (Floci-safe)

```
# We do NOT use aws-actions/configure-aws-credentials
(that action calls real AWS STS — rejects fake test/test credentials)

echo "AWS_ACCESS_KEY_ID=test" >> $GITHUB_ENV
echo "AWS_SECRET_ACCESS_KEY=test" >> $GITHUB_ENV
```

Step 3 — Build Images

```
for svc in user-service product-service ...; do
  cd $svc && mvn package -DskipTests -q && cd ..
  # Tag for Floci ECR
  docker build -t localhost:5100/ecommerce/$svc:$GIT_SHA .
  # Tag for docker-compose (image name = ecommerce-$svc)
  docker tag localhost:5100/ecommerce/$svc:latest ecommerce-$svc:latest
done
```

Images are tagged with the **git commit SHA** (`abc1234`).

This means every image is traceable back to the exact commit that produced it.

Step 4 — Trivy Security Scan

```
uses: aquasecurity/trivy-action@master
with:
  image-ref: localhost:5100/ecommerce/user-service:${{ steps.meta.outputs.version }}
  severity: CRITICAL,HIGH
  ignore-unfixed: true
  cache: true   # avoids re-downloading 100MB vulnerability DB each run
```

Scans one representative image (all 6 share the same `eclipse-temurin:21-jre-alpine` base).

`exit-code: 0` — scan reports findings but doesn't fail the build (informational).

Step 5 — Push to Floci ECR

```
aws ecr get-login-password --endpoint-url http://localhost:4566 | \
  docker login --username AWS --password-stdin localhost:5100


docker push localhost:5100/ecommerce/user-service:abc1234


docker push localhost:5100/ecommerce/user-service:latest
```

Step 6 — Smoke Test via docker-compose

```
# Start everything (no-build = uses images we just built)
docker compose up -d --no-build
```

Wait for ALL 3 services to be ready (not just one)

```
for i in $(seq 1 120); do
  P=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:8000/api/products)
  U=$(curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/api/users/login
...)
  O=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:8000/api/orders/user/1)
  if [ "$P" = "200" ] && [ "$U" != "503" ] && [ "$O" != "503" ]; then
    break
  fi
  sleep 5
done
```

```
Run smoke tests

curl -f -X POST http://localhost:8000/api/users/register ...
curl -f http://localhost:8000/api/products
curl -f -X POST http://localhost:8000/api/products ...
```

Why docker-compose and not K8s in CI? K8s (Kind) takes 5-10 minutes to start on a 2-CPU GitHub runner. docker-compose with pre-built images is ready in ~90 seconds. K8s manifests are tested locally with Floci EKS — CI verifies app logic.

Job 3: Production Deployment (main branch, approval required)

Same steps as Job 2 with one difference: `environment: production`.

This pauses the job at deployment time and requires a designated reviewer to click **"Approve deployment"** in GitHub:

```
GitHub → Actions → (failed run) → Review deployments → Approve and deploy
```

To set this up:

1. GitHub → Settings → Environments → New environment → `production`
2. Add required reviewers (e.g., tech lead)
3. Set wait timer: 5 minutes (prevents accidental approvals)

Dependabot (Automatic Dependency Updates)

File: `.github/dependabot.yml`

Every week, Dependabot opens PRs for:

- Maven dependencies (Spring Boot, Kafka, Redis client, etc.)
- Docker base images (`eclipse-temurin:21-jre-alpine`)
- GitHub Actions versions (`actions/checkout@v4` , etc.)

Each Dependabot PR runs the full test suite automatically. You merge it if tests pass.

Branch Protection Rules (Set Up in GitHub)

For `main`:

```
Settings → Branches → main
☑ Require PR before merging
☑ Require status checks: test
☑ Require approvals: 1
☑ Dismiss stale reviews on new commits
☑ Restrict force pushes
```

For `develop`:

```
☑ Require PR before merging
☑ Require status checks: test
☐ Approvals: optional (or 1 for teams)
```

Full Developer Workflow Summary

```
Feature work:
  git checkout -b feature/xxx develop
  [make changes + mvn test locally]
  git push → open PR to develop
  → CI: tests run (Job 1)
  → Team review
  → Merge to develop → CI: build + deploy to dev (Job 2)


Release:


  Open PR: develop → main
  → CI: tests run (Job 1)
  → Tech lead approves PR
  → Merge to main → CI: build + PAUSE for approval (Job 3)
  → Reviewer approves deployment
  → Deploy to production
```

Adding a New Service to CI

1. Add service to `SERVICES` env var in `.github/workflows/ci-cd.yml`
2. Add service folder with `pom.xml` and at least one unit test

3. Add `image: ecommerce-:latest` to `docker-compose.yml`
4. Add ECR repo creation to `scripts/03-ecr.sh`
5. Add K8s Deployment + HPA to `k8s/services/deployments.yaml`